第11章 查询处理深度详解

我将基于你的课件，**更深入、更全面**地解析每个重要概念，并提供 **丰富的实例和对比分析** 。

一、查询处理架构再透视

1.1 三层处理模型（课件第15.3-15.6页）

```
SQL查询
    ↓
解析器 → 语法分析树
    ↓
翻译器 → 关系代数表达式树
    ↓
优化器 → 标注了算法的执行计划树
    ↓
执行引擎 → 查询结果
```

关键洞察 ：

- **解析与翻译**是机械的，一对一映射
- **优化**是核心难点，但本章重点在 **执行算法** ，为优化提供基础
- **执行引擎**是"如何做"，优化器是"哪个更好"

二、选择操作的深度解析

2.1 线性搜索的代价细节

课件第15.9页公式 ：代价 = `b_r * t_T + 1 * t_S`

但这里隐藏了细节 ：

- `b_r`是关系的块数
- `t_T`≈ 0.1ms（假设磁盘传输速率100MB/s，块大小4KB）
- `t_S`≈ 4ms（7200RPM磁盘的平均寻道时间）

例题深化 ：

假设关系r有：

- 记录大小：200字节
- 块大小：4KB
- 记录数：100,000
- 块因子：20记录/块
- 总块数：5,000块

线性搜索代价：

磁盘I/O时间 = 5,000 × 0.1ms = 500ms

寻道时间 = 1 × 4ms = 4ms

总时间 ≈ 504ms

但注意 ：如果记录跨块存储，或存在空闲空间，实际块数可能更多！

2.2 索引扫描的复杂情况

情况1：等值查询，有主键索引

```
SELECT * FROM instructor WHERE ID = '10101';
```

- 索引：B+树，高度h=3
- 过程：
    1. 从根节点到叶节点：3次I/O
    2. 叶节点包含数据记录（聚簇索引）或记录指针
    3. 读取记录：1次I/O
- 总I/O = 4次

情况2：范围查询，聚簇索引

```
SELECT * FROM instructor WHERE salary BETWEEN 60000 AND 80000;
```

- 索引：B+树，聚簇
- 过程：
    1. 找到第一个salary=60000的叶节点：h次I/O
    2. 顺序扫描叶节点链表，直到salary>80000
    3. 读取数据块（连续存储）
- 假设匹配100个记录，存储在5个连续块
- 总I/O = h + 5 = 3 + 5 = 8次

情况3：复合条件

```
SELECT * FROM instructor
WHERE dept_name = 'Comp. Sci.' AND salary > 70000;
```

算法选择策略 ：

1. 如果 `(dept_name, salary)` 上有复合索引 → 使用索引
2. 如果只有 `dept_name`索引 → 先索引找到部门，再过滤salary
3. 如果只有 `salary`索引 → 先索引找到salary>70000，再过滤部门
4. 如果都无索引 → 线性扫描+过滤

例题：多条件选择的最优算法选择

表 `employee(id, name, dept, salary, age)` ，统计信息：

- 总记录数：100,000
- 块数：2,000
- 索引1：`dept`上的哈希索引（非聚簇）
- 索引2：`salary`上的B+树索引（非聚簇）
- 选择度：dept='IT'占5%，salary>80000占10%
- 条件独立性假设

查询：`SELECT * FROM employee WHERE dept='IT' AND salary>80000;`

比较以下策略：

1. 线性扫描
2. 使用dept索引
3. 使用salary索引
4. 交集位图（如果有）

解答 ：

1. **线性扫描** ：2,000次I/O

2. **使用dept索引**：通过哈希找到dept='IT'的记录（5,000条），每记录1次随机I/O → 5,000次I/O
3. **使用salary索引**：通过B+树找到salary>80000的记录（10,000条），每记录1次随机I/O → 10,000次I/O
4. **最优策略**：先用dept索引（5,000次I/O），然后在内存中过滤salary>80000（约500条满足）→ 5,000次I/O

三、排序算法的深入分析

3.1 外部归并排序的完整流程

课件第15.18-15.23页展示了外部排序，但可以更详细：

阶段0：初始归并段创建

```
输入：M=3个内存块，关系b=12块
过程：
1. 重复直到输入为空：
   a. 读入M块到内存
   b. 内部排序（如快速排序）
   c. 将排序结果写回磁盘
结果：ceil(12/3)=4个初始归并段，每个3块
I/O：读12块 + 写12块 = 24次I/O
```

阶段1：第一趟归并

```
假设内存M=3块，使用2路归并
过程：
1. 为每个归并段分配1个输入缓冲区（共2个），1个输出缓冲区
2. 归并段1和2 → 归并段5（6块）
3. 归并段3和4 → 归并段6（6块）
I/O：读12块 + 写12块 = 24次I/O
```

阶段2：第二趟归并

```
归并段5和6 → 最终有序关系（12块）
I/O：读12块 + 写12块 = 24次I/O
总I/O：24+24+24=72次I/O
```

3.2 多路归并的数学关系

重要公式：

- 归并的**趟数** = ⌈log_{M-1}(N)⌉
  - N = 初始归并段数 = ⌈b/M⌉
  - M = 内存块数
  - M-1 = 归并路数（1块输出缓冲区）

例题：计算最小趟数

给定b=1,000块，M=101块

初始归并段数 N = ⌈1000/101⌉ = 10

归并路数 = M-1 = 100

由于10 < 100，所以 **一趟归并即可**！

对比：如果M=11块

N = ⌈1000/11⌉ = 91

归并路数 = 10

趟数 = ⌈$\log_{10}(91)$⌉ = 2趟

四、连接操作的全面对比

4.1 嵌套循环连接变体

变体1：块嵌套循环连接优化

```
# 伪代码展示块嵌套循环连接
for each block B_r of relation R:  # 外层循环，每次读一块R
    read block B_r into memory
    for each block B_s of relation S:  # 内层循环，扫描整个S
        read block B_s into memory
        for each tuple r in B_r:
            for each tuple s in B_s:
                if condition(r, s):
                    add (r, s) to output
```

代价分析 ：

- 朴素：`n_r * b_s + b_r`次I/O
- 块优化：`b_r * b_s + b_r`次I/O
- 内存优化：`ceil(b_r/(M-2)) * b_s + b_r`次I/O

变体2：索引嵌套循环连接细节

```
# 伪代码展示索引嵌套循环连接
for each tuple r in relation R:  # 顺序扫描R
    # 使用S上的索引查找匹配的元组
    matching_s = index_lookup(S.index, r.join_key)
    for each tuple s in matching_s:  # 索引返回的结果
        if condition(r, s):
            add (r, s) to output
```

索引查找代价c的组成 ：

- B+树索引：h（高度）+ 1（数据块访问）
- 哈希索引：1（桶访问）+ 平均溢出链长度
- 聚簇索引：h + 连续块数
- 非聚簇索引：h + 匹配记录数（最坏情况）

4.2 归并连接的完整示例

课件第15.30-15.32页的扩展 ：

场景 ：

- 关系R：按join_key排序，b_R=100块
- 关系S：按join_key排序，b_S=50块
- 内存M=3块

归并过程 ：

```
初始化：
  buffer_R[1] ← 读取R的第1块
  buffer_S[1] ← 读取S的第1块
  output_buffer ← 空

归并循环：
  while buffer_R有元组 and buffer_S有元组:
    r ← buffer_R当前最小元组
    s ← buffer_S当前最小元组

    if r.key == s.key:
      # 连接匹配的元组
      output_buffer ← (r, s)的所有组合
      advance both buffers
    elif r.key < s.key:
      advance buffer_R
```

```
    else:
      advance buffer_S

    # 缓冲区空时加载下一块
    if buffer_R空 and R还有块:
      读取R的下一块到buffer_R
    if buffer_S空 and S还有块:
      读取S的下一块到buffer_S
```

代价 ：

- 如果已排序：`b_R + b_S = 100 + 50 = 150`次I/O
- 如果需要排序：`b_R + b_S + 2 *` 排序代价

4.3 哈希连接的深度技术细节

阶段1：分区（Partitioning）的数学原理

```
关系R有b_R=100块，S有b_S=50块，内存M=20块

目标：将S划分为若干分区，使得每个分区 ≤ M-1块
（M-1是因为需要1块作为输入缓冲区）

步骤：
1. 选择哈希函数h，将R和S按连接属性哈希到多个桶
2. 假设初始桶数k = ceil(b_S / (M-1)) = ceil(50/19) ≈ 3
3. 但实际中，为了处理数据倾斜，通常选择桶数k = ceil(b_S / (M-1)) * f
   （f是安全因子，通常1.2-1.5）

分区过程I/O代价：
  读R：b_R次
  写R分区：b_R次
  读S：b_S次
  写S分区：b_S次
  小计：2*(b_R + b_S) = 2*(100+50)=300次
```

阶段2：探测（Probing）的详细过程

```
对于每个分区i=1 to k:
  1. 将S的分区S_i读入内存：b_S_i次I/O
  2. 在内存中为S_i构建哈希表（使用h2，可能与h1不同）
  3. 读取R的分区R_i：b_R_i次I/O
  4. 对R_i中的每个元组r：
     a. 用h2哈希r的连接属性
     b. 在S_i的哈希表中查找匹配
     c. 如果找到，输出连接结果

探测阶段I/O代价：
  Σ(b_R_i + b_S_i) = b_R + b_S = 100+50=150次

总代价 = 300 + 150 = 450次I/O
```

递归分区 ：

如果某个分区S_i仍然大于M-1，则需要对该分区进行二次哈希。

例题：哈希连接的实际计算

关系R: 1,000块，10,000个元组

关系S: 500块，5,000个元组

内存M: 52块

连接属性：employee_id

计算哈希连接的代价。

解答 ：

1. **分区数计算** ：

- 目标：每个S分区 ≤ M-1 = 51块
- 最小分区数k = ceil(500/51) ≈ 10
- 选择k=10（2的幂次便于哈希）

1. **分区大小估计** ：

- 平均每个S分区大小 = 500/10 = 50块 ≤ 51块 ✓
- 平均每个R分区大小 = 1000/10 = 100块

1. **代价计算** ：

- 分区阶段：2*(1000+500) = 3,000次I/O
- 探测阶段：
  - 对每个分区i：读取S_i(50块)+R_i(100块)=150块
  - 总共10个分区：10 * 150=1,500次I/O
- 总I/O = 3,000 + 1,500 = 4,500次

1. **与块嵌套循环比较** ：

- 块嵌套循环（R为外表）：
  - 内存分配：M-2=50块给R
  - 组数 = ceil(1000/50)=20
  - 代价 = 1000 + 20 * 500 = 11,000次I/O
- 哈希连接明显更优！

五、复杂查询处理示例

5.1 多表连接优化

查询 ：

```
SELECT s.name, c.title, t.grade
FROM student s, takes t, course c
WHERE s.ID = t.ID
  AND t.course_id = c.course_id
  AND s.dept_name = 'Comp. Sci.'
  AND c.dept_name = 'Biology';
```

关系统计 ：

- student: 5,000个元组，100块
- takes: 10,000个元组，200块
- course: 500个元组，20块
- 内存M: 30块
- 索引：student(dept_name)有B+树索引

执行计划比较 ：

计划1：左深树，嵌套循环连接

```
首先：σ_dept='Comp. Sci.'(student) → 结果A（约500个元组，10块）
然后：A ⋈ takes（嵌套循环，A为外表）
最后：结果 ⋈ course
```

计划2：右深树，哈希连接

首先：σ_dept='Comp. Sci.'(student) → 结果A
同时：σ_dept='Biology'(course) → 结果B（约50个元组，2块）
然后：takes分别与A、B哈希连接

计划3：使用索引

1. 通过student索引找到dept='Comp. Sci.'的元组
2. 对于每个找到的student，通过takes索引找对应选课
3. 对于每个选课，通过course索引找课程信息

代价估算对比 ：

- 计划1：约 10 + 10 * 200 + 结果大小*20
- 计划2：约 10+2 + 2*(200+哈希连接代价)
- 计划3：索引高度+记录数*(索引高度+...)

结论 ：需要具体计算，但哈希连接通常对等值连接更优。

5.2 包含聚合的查询

查询 ：

```
SELECT dept_name, AVG(salary)
FROM instructor
GROUP BY dept_name
HAVING AVG(salary) > 70000;
```

执行策略 ：

策略1：排序分组

1. 按dept_name排序instructor
2. 扫描排序结果，对每个dept_name：
   a. 计算salary总和和计数
   b. 计算平均值
   c. 如果>70000，输出
代价：排序代价 + 扫描代价

策略2：哈希分组

1. 使用哈希函数h(dept_name)将元组分到桶
2. 对每个桶在内存中：
   a. 维护哈希表：dept_name → (sum, count)
   b. 对每个元组，更新对应组的sum和count
3. 扫描哈希表，计算平均值，过滤
代价：一次扫描 + 哈希计算

六、表达式求值的工程实现

6.1 物化与流水线的权衡

物化实现 ：

```
-- 查询：找出计算机系教师的平均工资
SELECT AVG(salary)
FROM instructor
WHERE dept_name = 'Comp. Sci.';
```

执行计划树 ：

```
γ_AVG(salary)
         |
     σ_dept='Comp. Sci.'
         |
     instructor
```

物化执行 ：

1. 执行σ：扫描instructor，输出中间结果Temp1
2. 物化Temp1到磁盘
3. 读取Temp1，计算AVG(salary)
4. 输出结果

流水线执行：

1. 扫描instructor时，立即检查dept_name
2. 如果匹配，将salary传递给聚合操作
3. 聚合操作维护运行总和和计数
4. 扫描完成时，计算平均值并输出

6.2 流水线的两种实现方式

需求驱动（拉取模型）：

```
class PipelineIterator:
    def __init__(self, child_op):
        self.child = child_op

    def next(self):
        # 从子操作"拉取"下一个元组
        tuple = self.child.next()
        if tuple is None:
            return None
        # 处理元组
        return self.process(tuple)
```

生产者驱动（推送模型）：

```
class PipelineOperator:
    def __init__(self, parent_op):
        self.parent = parent_op
        self.parent.set_consumer(self)

    def consume(self, tuple):
        # 父操作"推送"元组过来
        result = self.process(tuple)
        if result and self.consumer:
            self.consumer.consume(result)
```

七、真题分析与应试重点

根据你提供的真题，以下是**高频考点**：

7.1 连接算法选择（真题四.3类似）

题目形式：给定关系大小、内存大小、索引情况，选择最佳连接算法并计算代价。

解题模板：

1. 列出所有可行算法
2. 计算每个算法的I/O代价
3. 考虑实际因素（内存限制、索引类型）
4. 选择最优算法

例题：

R(A,B): 1,000块，S(B,C): 500块

连接：R ⋈ S

内存M=52块

索引：S在B上有B+树索引（高度3，聚簇）

比较：块嵌套循环、索引嵌套循环、哈希连接、归并连接

解答步骤 ：

1. **块嵌套循环** （R外表）：

- 内存分配：M-2=50块给R
- 组数=ceil(1000/50)=20
- 代价=1000 + 20×500 = 11,000

1. **索引嵌套循环** （R外表）：

- 假设R有10,000个元组
- 对每个R元组，索引查找S
- 索引查找代价：h+1=3+1=4（聚簇，假设匹配1个记录）
- 代价=1000 + 10000×4 = 41,000

1. **哈希连接** ：

- 分区数k=ceil(500/51)≈10
- 代价=2×(1000+500)+(1000+500)=4,500

1. **归并连接** ：

- 假设R,S已按B排序
- 代价=1000+500=1,500
- 如果需要排序：加上2×排序代价

结论 ：归并连接最优（如果已排序），否则哈希连接。

7.2 复杂查询优化（真题三类似）

解题技巧 ：

1. 识别可用的索引
2. 考虑选择下推
3. 估算中间结果大小
4. 选择连接顺序和算法
5. 考虑流水线机会

八、高级话题延伸

8.1 布隆过滤器优化哈希连接

在哈希连接中，可以使用**布隆过滤器**减少不必要的磁盘I/O：

```
阶段1：构建分区时，为每个S分区创建布隆过滤器
阶段2：探测R分区时，先用布隆过滤器过滤
  - 如果布隆过滤器说"不在"，则跳过该元组
  - 如果布隆过滤器说"可能在"，则继续探测
可以过滤掉大量不匹配的元组，减少I/O
```

8.2 基于排序的聚合优化

当内存不足时，基于排序的聚合也需要外部排序：

```
1. 按GROUP BY列排序
2. 扫描排序结果，相同组的元组连续
3. 对每组计算聚合函数
4. 应用HAVING条件过滤

代价 = 排序代价 + 扫描代价
```

九、常见陷阱与易错点

1. **忽略聚簇与非聚簇索引的区别** ：聚簇索引的匹配元组物理相邻，非聚簇索引可能每个元组在不同块
2. **忘记考虑输出缓冲区** ：内存分配时要为输出留1块
3. **错误估算分区数** ：哈希连接中，分区数应使每个分区≤M-1
4. **忽略数据倾斜** ：哈希连接在数据严重倾斜时性能下降
5. **流水线的误用** ：不是所有操作都适合流水线（如排序、哈希连接构建阶段）

十、练习题

1. 关系R(a,b,c)有10,000个元组，存储在500块中。在属性a上有B+树索引（高度2，非聚簇）。查询 `SELECT * FROM R WHERE a=10 AND b>20` ，假设：

   - a=10的选择度：1%
   - b>20的选择度：50%
   - 条件独立

   哪种执行计划最优？计算代价。

2. 设计一个包含排序、连接、分组的复杂查询，画出至少3种不同的执行计划树，并估算每种计划的代价。

3. 解释为什么在内存有限的情况下，哈希连接可能比归并连接更优，即使归并连接的理论I/O次数更少。

这些深度解析和示例应该能帮助你全面掌握第11章的内容。每个知识点都配有详细的计算和比较，帮助你应对考试中的各种题型。如果需要进一步解释某个特定算法或有其他问题，请随时提问。